Data Structures & Algorithms Programming lab
6.0CA107C01H

Manav Rachna International Institute of Research and Studies

School of Computer Applications

Department of Computer Applications

Submitted By	
Student Name	Ronit Singh
Roll No	26MCAP101N004
Programme	Master of Computer Applications
Semester	1stSemester
Section/Group	A
Department	Computer Applications
Batch	2026-28
Submitted To	
Faculty Name	Dr. Sachin Sharma

1. Insert a new element in array

Input-

```
  GNU nano 6.2          insert.py
n = int(input("Enter number of elements: "))
a = []
print("Enter", n, "elements:")
for i in range(n):
    a.append(int(input()))

pos = int(input("Enter position to insert (1-" + str(n + 1) + "): "))
val = int(input("Enter value to insert: "))

a.append(0)
for i in range(n - 1, pos - 2, -1):
    a[i + 1] = a[i]
a[pos - 1] = val

print("Array after insertion:")
print(*a)
```

Output-

```
ronit@ubuntu:~$ python3 insert.py
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50
Enter position to insert (1-6): 3
Enter value to insert: 25
Array after insertion:
10 20 25 30 40 50
ronit@ubuntu:~$
```

2. Delete an element from array

Input-

```
  GNU nano 6.2          delete.py
n = int(input("Enter number of elements: "))
a = []
print("Enter", n, "elements:")
for i in range(n):
    a.append(int(input()))

pos = int(input("Enter position to delete (1-" + str(n) + "): "))

if pos < 1 or pos > n:
    print("Invalid position")
else:
    print("Deleted element:", a[pos - 1])
    for i in range(pos - 1, n - 1):
        a[i] = a[i + 1]
    a.pop()
    print("Array after deletion:")
    print(*a)
```

Output-

```
ronit@ubuntu:~$ python3 delete.py
Enter number of elements: 5
Enter 5 elements:
10
20
30
40
50
Enter position to delete (1-5): 2
Deleted element: 20
Array after deletion:
10 30 40 50
ronit@ubuntu:~$
```

3. Find product of two matrix

Input-

```
  GNU nano 6.2          matmul.py
r1, c1 = map(int, input("Enter rows and columns of matrix A: ").split())
r2, c2 = map(int, input("Enter rows and columns of matrix B: ").split())

if c1 != r2:
    print("Multiplication not possible")
else:
    print("Enter elements of matrix A:")
    A = [list(map(int, input().split())) for i in range(r1)]
    print("Enter elements of matrix B:")
    B = [list(map(int, input().split())) for i in range(r2)]

    C = [[0] * c2 for i in range(r1)]
    for i in range(r1):
        for j in range(c2):
            for k in range(c1):
                C[i][j] += A[i][k] * B[k][j]

    print("Product of the matrices:")
    for row in C:
        print(*row, sep="\t")
```

Output-

```
ronit@ubuntu:~$ python3 matmul.py
Enter rows and columns of matrix A: 2 3
Enter rows and columns of matrix B: 3 2
Enter elements of matrix A:
1 2 3
4 5 6
Enter elements of matrix B:
7 8
9 10
11 12
Product of the matrices:
58      64
139     154
ronit@ubuntu:~$
```

4. Sum of diagonal and non diagonal elements in matrix

Input-

```
  GNU nano 6.2          diag.py
n = int(input("Enter order of square matrix: "))
print("Enter elements of matrix:")
a = [list(map(int, input().split())) for i in range(n)]

diag = 0
nondiag = 0
for i in range(n):
    for j in range(n):
        if i == j:
            diag += a[i][j]
        else:
            nondiag += a[i][j]

print("Sum of diagonal elements =", diag)
print("Sum of non-diagonal elements =", nondiag)
```

Output-

```
ronit@ubuntu:~$ python3 diag.py
Enter order of square matrix: 3
Enter elements of matrix:
1 2 3
4 5 6
7 8 9
Sum of diagonal elements = 15
Sum of non-diagonal elements = 30
ronit@ubuntu:~$
```

5. Enter marks of 5 students for 3 subjects each and find the percentage of students

Input-

```
  GNU nano 6.2          marks.py
marks = []
for i in range(5):
    row = list(map(int, input("Enter marks of student " + str(i + 1) + " in 3 subjects: ").split()))
    marks.append(row)

print("\nStudent\tTotal\tPercentage")
for i in range(5):
    total = sum(marks[i])
    percentage = total / 300 * 100
    print(f"{i + 1}\t{total}\t{percentage:.2f}%")
```

Output-

```
ronit@ubuntu:~$ python3 marks.py
Enter marks of student 1 in 3 subjects: 78 85 92
Enter marks of student 2 in 3 subjects: 65 70 58
Enter marks of student 3 in 3 subjects: 90 88 95
Enter marks of student 4 in 3 subjects: 45 52 60
Enter marks of student 5 in 3 subjects: 72 68 80

Student Total   Percentage
1       255     85.00%
2       193     64.33%
3       273     91.00%
4       157     52.33%
5       220     73.33%
ronit@ubuntu:~$
```

6. Stack

(a) Stack operations

Input-

GNU nano 6.2 stack_ops.py

```
MAX = 5

class Stack:
    def __init__(self):
        self.items = []

    def push(self, x):
        if len(self.items) == MAX:
            print("Stack Overflow")
        else:
            self.items.append(x)
            print(x, "pushed")

    def pop(self):
        if len(self.items) == 0:
            print("Stack Underflow")
        else:
            print(self.items.pop(), "popped")

    def peek(self):
        if len(self.items) == 0:
            print("Stack is empty")
        else:
            print("Top element =", self.items[-1])

    def display(self):
        if len(self.items) == 0:
            print("Stack is empty")
        else:
            print("Stack elements:", *reversed(self.items))

s = Stack()
while True:
    print("\n1.Push 2.Pop 3.Peek 4.Display 5.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        s.push(int(input("Enter value: ")))
    elif ch == 2:
        s.pop()
    elif ch == 3:
        s.peek()
    elif ch == 4:
        s.display()
    elif ch == 5:
        break
    else:
        print("Invalid choice")
```

Output-

```
ronit@ubuntu:~$ python3 stack_ops.py

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 10
10 pushed

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 20
20 pushed

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 1
Enter value: 30
30 pushed

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack elements: 30 20 10

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 3
Top element = 30

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
30 popped

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 4
Stack elements: 20 10

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
20 popped

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
10 popped

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 2
Stack Underflow

1.Push 2.Pop 3.Peek 4.Display 5.Exit
Enter choice: 5
ronit@ubuntu:~$
```

(b) Reverse of a string using stack

Input-

```
  GNU nano 6.2          rev.py
text = input("Enter a string: ")

stack = []
for ch in text:
    stack.append(ch)

reversed_text = ""
while len(stack) > 0:
    reversed_text += stack.pop()

print("Reversed string:", reversed_text)
```

Output-

```
ronit@ubuntu:~$ python3 rev.py
Enter a string: Ronit
Reversed string: tinoR
ronit@ubuntu:~$
```

(c) Parenthesis matching

Input-

```
  GNU nano 6.2          paren.py
pairs = {')': '(', '}': '{', ']': '['}

exp = input("Enter an expression: ")
stack = []
balanced = True

for ch in exp:
    if ch in "({[":
        stack.append(ch)
    elif ch in ")}]":
        if len(stack) == 0 or stack.pop() != pairs[ch]:
            balanced = False
            break

if len(stack) != 0:
    balanced = False

if balanced:
    print("Parentheses are balanced")
else:
    print("Parentheses are NOT balanced")
```

Output-

```
ronit@ubuntu:~$ python3 paren.py
Enter an expression: {a+(b*c)-[d/e]}
Parentheses are balanced
ronit@ubuntu:~$
```

(c) Parenthesis matching (unbalanced input)

Input-

```
  GNU nano 6.2          paren.py
pairs = {')': '(', '}': '{', ']': '['}

exp = input("Enter an expression: ")
stack = []
balanced = True

for ch in exp:
    if ch in "({[":
        stack.append(ch)
    elif ch in ")}]":
        if len(stack) == 0 or stack.pop() != pairs[ch]:
            balanced = False
            break

if len(stack) != 0:
    balanced = False

if balanced:
    print("Parentheses are balanced")
else:
    print("Parentheses are NOT balanced")
```

Output-

```
ronit@ubuntu:~$ python3 paren.py
Enter an expression: {a+(b*c]-d}
Parentheses are NOT balanced
ronit@ubuntu:~$
```

7. Queue

(a) General queue operations

Input-

GNU nano 6.2 queue.py

```
MAX = 5

class Queue:
    def __init__(self):
        self.q = [None] * MAX
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if self.rear == MAX - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear += 1
            self.q[self.rear] = x
            print(x, "inserted")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            print(self.q[self.front], "deleted")
            self.front += 1

    def display(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue is empty")
        else:
            print("Queue elements:", *self.q[self.front:self.rear + 1])

q = Queue()
while True:
    print("\n1.Enqueue 2.Dequeue 3.Display 4.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        q.enqueue(int(input("Enter value: ")))
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.display()
    elif ch == 4:
        break
    else:
        print("Invalid choice")
```

Output-

```
ronit@ubuntu:~$ python3 queue.py

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 20
20 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 40
40 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 50
50 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 60
Queue Overflow

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 3
Queue elements: 10 20 30 40 50

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 2
10 deleted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 2
20 deleted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 3
Queue elements: 30 40 50

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 4
ronit@ubuntu:~$
```

(b) Circular queue operations

Input-

GNU nano 6.2 cqueue.py

```
MAX = 5

class CircularQueue:
    def __init__(self):
        self.cq = [None] * MAX
        self.front = -1
        self.rear = -1

    def enqueue(self, x):
        if (self.rear + 1) % MAX == self.front:
            print("Circular Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear = (self.rear + 1) % MAX
            self.cq[self.rear] = x
            print(x, "inserted")

    def dequeue(self):
        if self.front == -1:
            print("Circular Queue Underflow")
        else:
            print(self.cq[self.front], "deleted")
            if self.front == self.rear:
                self.front = self.rear = -1
            else:
                self.front = (self.front + 1) % MAX

    def display(self):
        if self.front == -1:
            print("Circular Queue is empty")
        else:
            items = []
            i = self.front
            while True:
                items.append(self.cq[i])
                if i == self.rear:
                    break
                i = (i + 1) % MAX
            print("Circular Queue elements:", *items)

q = CircularQueue()
while True:
    print("\n1.Enqueue 2.Dequeue 3.Display 4.Exit")
    ch = int(input("Enter choice: "))
    if ch == 1:
        q.enqueue(int(input("Enter value: ")))
    elif ch == 2:
        q.dequeue()
    elif ch == 3:
        q.display()
    elif ch == 4:
        break
    else:
        print("Invalid choice")
```

Output-

```
ronit@ubuntu:~$ python3 cqueue.py

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 10
10 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 20
20 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 30
30 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 40
40 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 50
50 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 60
Circular Queue Overflow

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 2
10 deleted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 2
20 deleted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 1
Enter value: 60
60 inserted

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 3
Circular Queue elements: 30 40 50 60

1.Enqueue 2.Dequeue 3.Display 4.Exit
Enter choice: 4
ronit@ubuntu:~$
```